

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	Ankala Lakshmi
Name	192373066

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 25

Code of Conduct

I A. Lakshmi(192373066) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:*Lakshmi*

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

1.A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.

Problem

A Hadoop job repeatedly fails because the input data is divided into **uneven input splits**. At the same time, some Reducers remain idle while other tasks are still processing.

Likely Design Error

The main design error is **poor data partitioning and data skew**.

Hadoop normally divides input data into splits and assigns them to Mapper tasks. If some splits are much larger than others, the corresponding Mapper tasks take much longer.

For example:

- Mapper 1 → 100 MB
- Mapper 2 → 100 MB
- Mapper 3 → 100 MB
- Mapper 4 → 5 GB

Mapper 4 becomes a bottleneck while the other Mappers finish early.

This is called **straggler behavior**.

Reducers may remain idle because they cannot receive enough data until the Mapper phase produces its output.

Other Possible Causes

- Very large files mixed with many small files.
- Poorly chosen block/split size.
- Data skew caused by certain keys occurring much more frequently.
- Poor partitioner design.
- Too few Mapper tasks.
- An inappropriate number of Reducers.

Corrective Actions

1. Balance Input Splits

Use an appropriate HDFS block size and input format so that input data is divided into reasonably balanced chunks.

2. Avoid Small and Irregular Files

Combine many small files when appropriate instead of processing thousands of tiny files individually.

3. Handle Data Skew

If one key occurs much more frequently than others, use a better partitioning strategy.

For example, instead of sending almost all records for one key to a single Reducer, distribute the workload more evenly.

4. Use an Appropriate Number of Reducers

Too few Reducers can create bottlenecks, while too many can create unnecessary overhead.

The number of Reducers should match the dataset size and available cluster resources.

5. Use a Combiner

A Combiner can perform local aggregation before data reaches the Reducers.

This reduces:

- Network traffic.
- Shuffle volume.
- Reducer workload.

6. Enable Speculative Execution When Appropriate

If one task is unusually slow because of a slow node, Hadoop can run another copy of the task on a different node.

2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.

Problem

An HDFS deployment experiences data unavailability whenever one DataNode fails.

Normally, HDFS is designed to tolerate individual DataNode failures through data replication. Therefore, frequent unavailability after a single node failure indicates a **replication or NameNode high-availability design problem**.

Likely Replication Error

The HDFS replication factor may be too low.

For example, if the replication factor is:

1

then each block has only one copy.

If that DataNode fails:

The only copy of the block becomes unavailable.

A better configuration is usually a replication factor of **3** for important production data, subject to storage constraints.

Another Replication Problem

Even when the replication factor is 3, the replicas might be placed poorly.

For example, all three copies could accidentally be concentrated within the same failure domain.

If that failure domain becomes unavailable, all copies may be affected.

Therefore, **rack awareness** should be configured so that replicas are distributed across different racks/failure domains.

Possible NameNode Design Error

The deployment may have only a **single NameNode**.

The NameNode manages HDFS metadata. If the NameNode fails, clients may be unable to access HDFS even though the actual data blocks still exist on DataNodes.

This is a classic **single point of failure**.

Corrective Actions

1. Increase Replication

For critical data, use an appropriate replication factor, commonly **3**.

2. Enable Rack Awareness

Place replicas across different racks/failure domains.

3. Configure NameNode High Availability

Use:

- Active NameNode.
- Standby NameNode.
- JournalNodes.
- Automatic failover.

4. Monitor Under-Replicated Blocks

After a DataNode failure, HDFS should automatically create replacement replicas on healthy DataNodes.

Administrators should monitor under-replicated blocks.

5. Maintain Sufficient Cluster Capacity

If the cluster has too few DataNodes, HDFS may not have enough healthy locations to recreate lost replicas.

3.A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.

Problem

A data-ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics database or data store.

Likely Causes

The main problem is usually a lack of **idempotency and duplicate-detection logic**.

If the same record is processed twice, the destination should recognize that it has already been inserted.

Cause 1: NiFi Flow Reprocessing

NiFi may retry or replay a record when:

- A downstream system fails.
- A connection is interrupted.
- A processor is restarted.
- A transaction fails.

If the destination simply inserts the record again, a duplicate is created.

Cause 2: No Unique Identifier

If records do not have a unique business key, the analytics store cannot reliably determine whether two records represent the same entity.

For example:

Customer ID = 1001

should identify a unique customer.

Cause 3: ETL Job Reprocessing

An ETL process may accidentally process the same input files or database records more than once.

For example:

- Monday's data is processed.
- The ETL job fails near the end.
- The entire Monday dataset is processed again.
- Existing records are inserted a second time.

Cause 4: Full Loads Instead of Incremental Loads

Repeatedly copying the entire source dataset can create duplicates if the destination does not replace or merge existing records.

Cause 5: Poor Checkpointing

If the ingestion system does not remember the last successfully processed record/file, it may start from an earlier position after recovery.

Corrective Actions

1. Use Unique Business Keys

Every record should have a reliable unique identifier.

For example:

Transaction_ID

or:

Customer_ID + Transaction_Date + Sequence_Number

2. Implement Idempotent Processing

Processing the same record multiple times should produce the same final result rather than creating additional copies.

3. Use Upsert/Merge Operations

Instead of always performing:

INSERT

use:

INSERT if new, UPDATE if already exists

This prevents duplicate records.

4. Use Incremental Ingestion

Only process:

- New records.
- Modified records.
- Deleted records where applicable.

For databases, Change Data Capture can be used.

5. Use NiFi Provenance and State

NiFi's provenance and state-management capabilities can help track what has been processed and support controlled recovery.

6. Deduplicate Before Loading

A transformation stage can identify duplicate records based on:

- Business key.
- Timestamp.
- File name.
- Record hash.

7. Make ETL Jobs Restartable

The ETL pipeline should resume from the last successful checkpoint instead of blindly processing the entire dataset again.

Conclusion

The main design error is **lack of idempotent ingestion and duplicate-control mechanisms**. The solution is to use unique keys, incremental ingestion, checkpointing, deduplication, and destination-side upsert/merge operations.

4.A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.

Problem

Multiple users submit jobs simultaneously, and some applications receive little or no resources.

This is called **resource starvation**.

Likely Scheduling Error

The cluster may be using a poorly configured scheduler or a single queue where one user's large job consumes most of the available resources.

For example:

- User A submits a large ML job.
- The job consumes most CPU and memory.
- Users B and C submit smaller jobs.
- Their applications wait for resources.

The problem is not necessarily that the cluster lacks resources. It may be that resources are **not being managed fairly**.

Common Causes

- No separate queues.
- Poor queue configuration.
- Incorrect resource limits.

- One user/application consuming excessive resources.
- No fair scheduling policy.
- No application or user limits.
- Poorly configured Capacity Scheduler or Fair Scheduler.

Corrective Approach

1. Use a Queue-Based Scheduler

A **Capacity Scheduler** is suitable for environments where different teams or workloads need guaranteed resources.

For example:

Queue	Purpose	Capacity
ETL	Data processing	40%
Reporting	Business reports	30%
ML	Machine learning	30%

This prevents one workload from consuming the entire cluster.

2. Configure User Limits

Set limits on how much of a queue a single user can consume.

For example, one user should not be able to occupy all containers in a queue.

3. Set Application Limits

Limit the maximum resources that a single application can request.

This prevents one huge application from starving other applications.

4. Use Fair Scheduling

Resources should be distributed fairly among active users and applications.

If three users submit jobs, the scheduler should not allow the first user's job to permanently consume the entire cluster.

5. Use Priorities Carefully

Important workloads can receive higher priority.

However, priority should not be configured so aggressively that lower-priority workloads never receive resources.

6. Use Preemption Carefully

If a high-priority queue needs resources and another queue is using resources beyond its guaranteed capacity, controlled preemption can return some resources.

However, excessive preemption can reduce overall cluster performance.

7. Monitor Cluster Resources

Monitor:

- CPU utilization.
- Memory utilization.
- Queue utilization.
- Application wait time.
- Container allocation.
- User resource consumption.

5.A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Problem

After a failure, the distributed storage system is recovered using a backup, but the restored data is older than the latest available data.

This indicates that the backup strategy is not properly synchronized with the production data.

Probable Design Flaw

The most likely problem is that the system relies on an **old or infrequently updated backup**.

For example:

- Monday: Backup created.
- Tuesday: Large amounts of new data added.
- Wednesday: System fails.
- Recovery uses Monday's backup.

The result is loss of Tuesday's and Wednesday's changes.

This is known as **stale backup data**.

Important Difference: Replication vs Backup

Replication and backup serve different purposes.

Replication

Replication protects against hardware/node failures.

For example, HDFS may maintain multiple copies of the same block.

If one DataNode fails, another replica can provide the data.

Backup

Backup protects against:

- Accidental deletion.
- Corruption.
- Human mistakes.
- Major cluster failures.
- Disaster.

Replication alone is not a complete backup strategy because a corrupted or accidentally deleted file may be propagated to all replicas.

Other Possible Design Errors

1. Infrequent Backups

Backups may be performed too rarely.

2. No Incremental Backup

Only full backups may be performed, making it difficult to capture recent changes efficiently.

3. No Recovery Point Objective

The organization may not have defined how much recent data it can afford to lose.

For example, if the required RPO is 15 minutes, a daily backup is clearly inadequate.

4. Backup Verification Missing

A backup may exist but not be tested regularly.

An untested backup cannot be assumed to be recoverable.

5. Incorrect Backup Selection

During recovery, the system may restore an older backup even though a newer valid backup exists.

Corrective Actions

1. Use Regular Incremental Backups

Capture changes frequently instead of relying only on occasional full backups.

2. Perform Periodic Full Backups

Use a combination of:

- Full backups.
- Incremental backups.

This balances recovery reliability and storage cost.

3. Define RPO

Recovery Point Objective defines the maximum acceptable amount of data loss.

For example:

RPO = 15 minutes

means the backup/recovery system should aim to lose no more than approximately 15 minutes of updates.

4. Define RTO

Recovery Time Objective defines how quickly the system must be restored.

For example:

RTO = 1 hour

means the organization expects the service to be restored within approximately one hour.

5. Maintain Backup Metadata

Maintain:

- Backup timestamp.
- Backup version.
- Data version.
- Source.
- Integrity status.

This ensures that recovery uses the correct and latest valid backup.

6. Test Backups Regularly

Perform recovery tests to verify that backups are:

- Complete.
- Consistent.
- Readable.
- Actually restorable.

7. Use Separate Backup Storage

Keep backups separate from the primary storage environment.

This protects against a complete cluster failure or disaster.

Quick Revision Table

Problem	Likely Error	Main Fix
Uneven MapReduce splits	Data skew / poor partitioning	Balance splits, improve partitioning, use Combiner
HDFS unavailable after one node fails	Low replication / poor replica placement / single NameNode	Replication, rack awareness, NameNode HA

NiFi/ETL duplicates	No idempotency/checkpointing	Unique keys, deduplication, upsert, incremental ingestion
YARN starvation	Poor queue/scheduler configuration	Capacity/Fair scheduling, queue limits, user limits
Outdated backup restored	Stale/infrequent backup	Incremental backups, versioning, RPO/RTO, recovery testing

Key Idea to Remember

For these five troubleshooting questions, remember the pattern:

Uneven processing → Partitioning problem

HDFS failure → Replication/HA problem

Duplicate data → Idempotency/CDC problem

YARN starvation → Scheduling/resource allocation problem

Old recovery data → Backup/RPO problem

These five keywords make the answers much easier to recall in an exam.